

Report on Problem 2

December 4, 2025

1 Introduction

Lexical analysis, also known as scanning or tokenization, is the first phase of a compiler where the source code is broken down into a sequence of tokens. These tokens represent the basic elements of the programming language, such as keywords, identifiers, literals, operators, and punctuation symbols. The lexical analyzer (lexer) reads the input stream of characters and groups them into meaningful tokens while ignoring whitespace, comments, and other irrelevant parts.

Syntax analysis, or parsing, follows lexical analysis and involves checking whether the sequence of tokens forms a valid sentence according to the grammar rules of the language. It constructs a parse tree or abstract syntax tree (AST) that represents the hierarchical structure of the program. Tools like Bison are used to generate parsers based on context-free grammars.

Semantic analysis is the phase that checks the meaning of the syntactically correct program. It ensures that the program adheres to the semantic rules of the language, such as type compatibility, variable declarations before use, and scope resolution. This phase often involves building and querying a symbol table to track declarations and their attributes.

In this report, we present a solution to a problem that demonstrates all three analyses using Flex (for lexical analysis) and Bison (for syntax and semantic analysis).

2 Problem Description

Consider the following code snippet:

```
def var as INTEGER = 0;
do
var = var++
loop until var < 10.0;
```

a) Perform Lexical Analysis on the given code snippet. b) Perform Syntax Analysis on the given code snippet. c) Perform Semantic Analysis on the given code snippet.

2.1 Lexical Analyzer (Flex File)

```
%option noyywrap

%{
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "p.tab.h"
%}

alpha [A-Za-z]
digit [0-9]
```

```

number {digit}+
float {digit}+\.{digit}+
ws [ \t\n]+
identifier {alpha}({alpha}|{digit})*

%%

{ws}                                { /* ignore whitespace */ }
"def"                               { yylval.str = strdup(yytext); return DEF; }
"as"                                { yylval.str = strdup(yytext); return AS; }
"INTEGER"                           { yylval.str = strdup(yytext); return INTEGER
; }
"do"                                { yylval.str = strdup(yytext); return DO; }
"<"                                { yylval.str = strdup(yytext); return LT; }
"loop"                              { yylval.str = strdup(yytext); return LOOP; }
"until"                             { yylval.str = strdup(yytext); return UNTIL;
}
"++"                                { yylval.str = strdup(yytext); return INC; }
"="                                 { yylval.str = strdup(yytext); return EQUAL;
}
";"                                 { yylval.str = strdup(yytext); return
    SEMICOLON; }
{number}                            { yylval.str = strdup(yytext); return NUMBER;
}
{float}                             { yylval.str = strdup(yytext); return FLOAT;
}
{identifier}                        { yylval.str = strdup(yytext); return ID; }

%%

```

2.2 Parser (Bison File)

```

%{
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include "semtab.h"
void yyerror(const char *s);
int yylex();
extern int yylineno;
extern char* yytext;
%}

%union {
    char* str;
}

%token <str> DEF AS INTEGER DO LT LOOP UNTIL INC EQUAL SEMICOLON NUMBER
    FLOAT ID
%start program

%%

program: declaration loop_stmt
        | declaration
        | loop_stmt
        ;

```

```

declaration: DEF ID AS INTEGER EQUAL NUMBER SEMICOLON
{
    printf("Semantic_Analysis_starts:\n");
    printf("Variable_declaration:\n");
    printf("Variable:\n%s, Type: INTEGER, Value:\n%s\n", $2,
        $6);
    if(symbol_table.lookup($2)) {
        printf("Semantic_Error: Variable '%s' already
            declared\n", $2);
    } else {
        symbol_table.insert($2, $4, $6);
        printf("Variable '%s' added to symbol_table\n", $2);
    }
}
;

loop_stmt: DO assignment LOOP UNTIL condition
{
    printf("Loop_statement_parsed\n");
}
;

assignment: ID EQUAL ID INC
{
    printf("Assignment_with_increment\n");
    printf("Assigning %s = %s++\n", $1, $3);
    if(!symbol_table.lookup($1)) {
        printf("Semantic_Error: Variable '%s' not declared\n",
            $1);
    }
    if(!symbol_table.lookup($3)) {
        printf("Semantic_Error: Variable '%s' not declared\n",
            $3);
    }
    if(symbol_table.lookup($1) && symbol_table.lookup($3)) {
        string type1 = symbol_table.getType($1);
        string type2 = symbol_table.getType($3);
        if(type1 != type2) {
            printf("Semantic_Warning: Type mismatch in
                assignment\n");
        }
    }
}
;

condition: ID LT FLOAT SEMICOLON
{
    printf("Condition_check\n");
    printf("Condition: %s < %s\n", $1, $3);
    if(!symbol_table.lookup($1)) {
        printf("Semantic_Error: Variable '%s' not declared\n",
            $1);
    }
    if(symbol_table.lookup($1)) {
        string varType = symbol_table.getType($1);
        if(varType == "INTEGER") {

```

```

        printf("Semantic Warning: Comparing INTEGER with
               FLOAT\n");
    }
}
;

%%

int main(void){

    printf("=== PARSING PHASE ===\n");
    yyparse();
    printf("=== PARSING COMPLETED ===\n\n");

    printf("=== SYMBOL TABLE ===\n");
    symbol_table.display();

    return 0;
}

void yyerror(const char *s)
{
    fprintf(stderr, "Syntax Error: %s at line %d\n", s, yylineno);
    exit(1);
}

```

2.3 Input (Input.txt)

```

def var as INTEGER = 0;
do
    var = var++
loop until var < 10.0;

```

2.4 Output (Output.txt)

```

=== PARSING PHASE ===
Semantic Analysis starts:
Variable declaration: Variable: var, Type: INTEGER, Value: 0
Variable 'var' added to symbol table
Assignment with increment
Assigning var = var++
Condition check
Condition: var < 10.0
Semantic Warning: Comparing INTEGER with FLOAT
Loop statement parsed
=== PARSING COMPLETED ===

=== SYMBOL TABLE ===

Symbol Table
Name          Type      Value
var           INTEGER    0

```

3 Conclusion

This report demonstrated the application of lexical, syntax, and semantic analysis through a practical coding problem. Using Flex and Bison, we tokenized, parsed, and semantically analyzed a custom code snippet. The solution highlights the importance of these compiler phases in ensuring correct program structure and meaning, particularly in type checking and variable declaration verification. Future work could extend these to more complex languages or incorporate optimization phases.